

Recruitment Job Fair Management System

Project Overview

The **Recruitment Job Fair Management System** is designed to streamline university-level job fairs by connecting **students**, **companies**, and **administrators** through mobile and web platforms.

Key Interfaces:

- **Student Mobile App:** View companies, apply for interviews, manage queues, accept/reject interview requests, receive notifications.
- **Company Web Panel:** Manage profile, post jobs, view students, call students for interviews, conduct interviews, submit surveys.
- **Admin Web Panel:** Manage rooms, pre-registered and on-spot companies, monitor interviews, generate reports, and collect survey responses.

The system ensures **queue management**, **conflict resolution**, **attendance verification**, and **real-time notifications**.

Key Features

Student Mobile App

- **Authentication:** Registration number + password sent via university email.
- **Profile:** Update CV, FYP title, FYP demo URLs, skills, CGPA, department, and FCM token.
- **Interview Queue:** View companies calling them for interviews with estimated times.
- **Interview Requests:** Students can **request interviews** and **accept or reject requests** from companies.
- **Push Notifications:** Receive alerts about interview calls, status changes, or announcements.

Company Web Panel

- **Profile:** Logo, description, number of representatives, industry.
- **Job Postings:** Job title, job type (full-time, internship, part-time, remote), description (optional), required skills.
- **Student Browsing:** Filter students by department, skills, FYP title, CGPA, CV, and demo video.
- **Interview Management:**

- Call students for interviews (students **cannot reject company-initiated calls**).
- Start and stop interviews.
- Mark students as **shortlisted, hired, or rejected**.
- Queue management with estimated interview times.
- **Survey Submission:** Post-event feedback via multiple-choice and descriptive questions.
- **Attendance Verification:** Pre-registered companies confirm arrival with **QR + secret key**, room is then allotted.

Admin Web Panel

- **Room Management:** Add rooms, set capacity, track status (vacant, tentatively allotted, allotted).
- **Company Management:**
 - Handle pre-registered and on-spot companies.
 - Allocate rooms automatically based on number of representatives.
- **Interview Monitoring:** Track interview counts, shortlisted/hired students, queue progress.
- **Survey & Reports:** View company survey responses, generate performance reports.

Workflow

1. **Pre-registration:** Companies register before the event; rooms are tentatively allocated. Companies can shortlist students they wish to interview.
2. **On-spot Registration:** Companies arriving at the event are assigned rooms; number of representatives and interview duration is recorded.
3. **Interview Scheduling:**
 - Companies start/stop interviews.
 - Students are called or can request interviews.
 - Conflict resolution prevents overlapping student interviews.
 - Students can **accept or reject requests**, except when called directly by companies.

4. Interview Status:

- Shortlisted for Interview (pre-registered)
- Shortlisted for Further Processing
- Hired
- Rejected

5. **Post-event:** Companies submit surveys; admins generate reports; students and companies receive certificates/notifications.

Database Schema

1. Users

```
CREATE TABLE Users (  
  user_id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role ENUM('admin','student','company') NOT NULL,  
  full_name VARCHAR(255),  
  phone VARCHAR(20),  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. Students

```
CREATE TABLE Students (  
  student_id SERIAL PRIMARY KEY,  
  user_id INT REFERENCES Users(user_id) ON DELETE CASCADE,  
  registration_no VARCHAR(50) UNIQUE NOT NULL,  
  cgpa DECIMAL(3,2),  
  cv_url TEXT,  
  fyp_title VARCHAR(255),
```

```
fyp_demo_url TEXT,  
skills TEXT[],  
department VARCHAR(100),  
fcm_token TEXT,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Companies

```
CREATE TABLE Companies (  
    company_id SERIAL PRIMARY KEY,  
    user_id INT REFERENCES Users(user_id) ON DELETE CASCADE,  
    name VARCHAR(255) NOT NULL,  
    logo_url TEXT,  
    description TEXT,  
    reps_count INT DEFAULT 1,  
    interview_duration_minutes INT,  
    industry VARCHAR(100),  
    arrival_status ENUM('pre_registered','on_spot') DEFAULT 'on_spot',  
    is_present BOOLEAN DEFAULT FALSE,  
    secret_key VARCHAR(100),  
    key_used BOOLEAN DEFAULT FALSE,  
    key_expiry TIMESTAMP,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4. Jobs

```
CREATE TABLE Jobs (  
    job_id SERIAL PRIMARY KEY,
```

```
company_id INT REFERENCES Companies(company_id) ON DELETE CASCADE,  
job_title VARCHAR(255) NOT NULL,  
job_type ENUM('full_time','internship','part_time','remote') NOT NULL,  
job_description TEXT,  
required_skills TEXT[],  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5. Rooms

```
CREATE TABLE Rooms (  
    room_id SERIAL PRIMARY KEY,  
    room_name VARCHAR(100) UNIQUE NOT NULL,  
    capacity INT,  
    status ENUM('vacant','tentatively_alloted','alloted') DEFAULT 'vacant',  
    company_id INT REFERENCES Companies(company_id),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

6. Interview Requests

```
CREATE TABLE InterviewRequests (  
    request_id SERIAL PRIMARY KEY,  
    company_id INT REFERENCES Companies(company_id) ON DELETE CASCADE,  
    student_id INT REFERENCES Students(student_id) ON DELETE CASCADE,  
    requested_by ENUM('company','student') NOT NULL,  
    status ENUM('pending','accepted','rejected') DEFAULT 'pending', -- student can reject  
    reason_for_reject TEXT, -- optional, if student rejects  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

);

7. Interviews

```
CREATE TABLE Interviews (  
    interview_id SERIAL PRIMARY KEY,  
    company_id INT REFERENCES Companies(company_id) ON DELETE CASCADE,  
    student_id INT REFERENCES Students(student_id) ON DELETE CASCADE,  
    request_id INT REFERENCES InterviewRequests(request_id),  
    status ENUM('queued','in_progress','shortlisted','hired','rejected') DEFAULT 'queued',  
    scheduled_time TIMESTAMP,  
    duration_minutes INT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

8. Notifications

```
CREATE TABLE Notifications (  
    notification_id SERIAL PRIMARY KEY,  
    user_id INT REFERENCES Users(user_id) ON DELETE CASCADE,  
    type VARCHAR(50),  
    payload JSONB,  
    is_read BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

9. Surveys

```
CREATE TABLE Surveys (  
    survey_id SERIAL PRIMARY KEY,  
    company_id INT REFERENCES Companies(company_id) ON DELETE CASCADE,  
    responses JSONB,  
    submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

);

10. Audit Logs

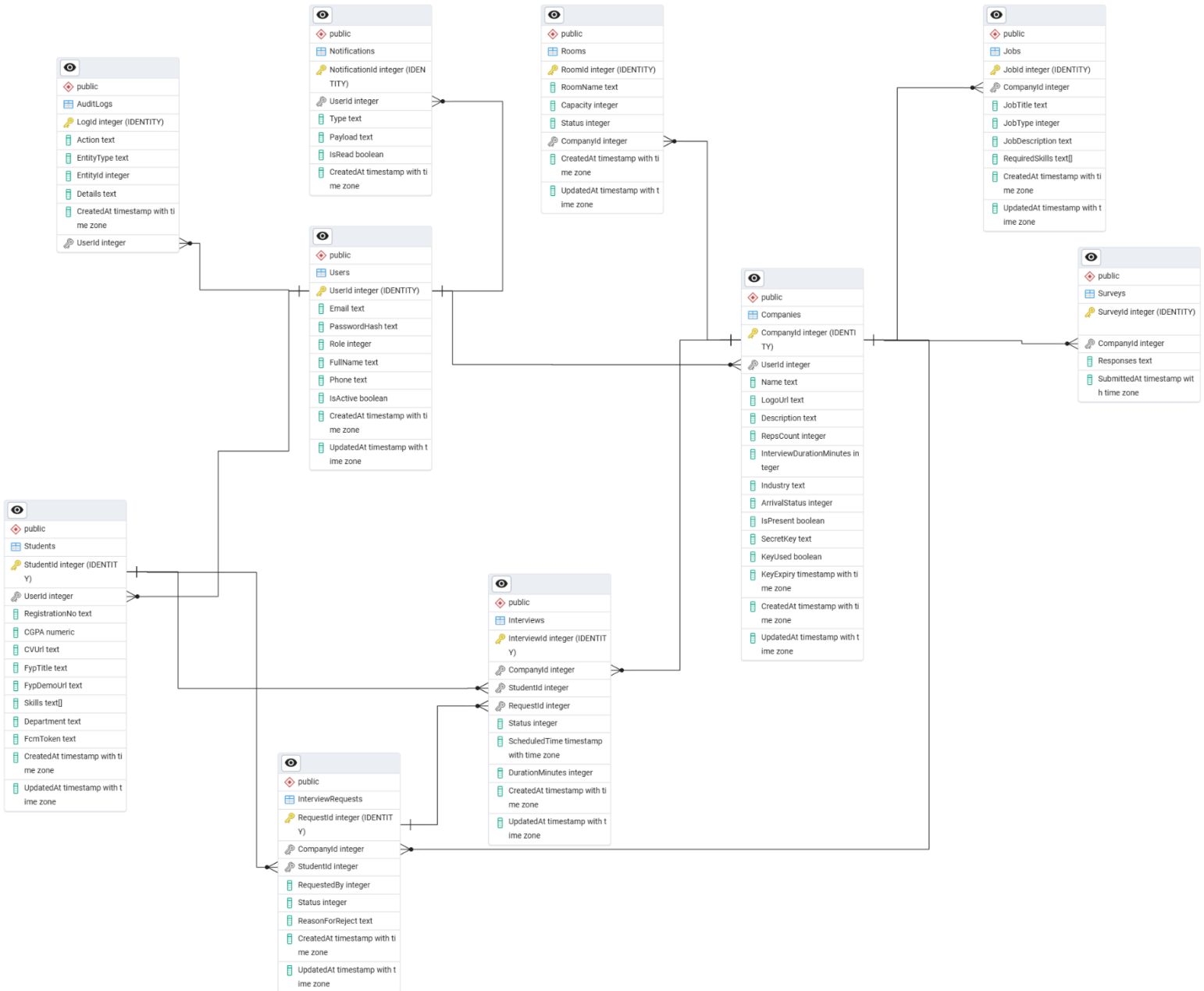
```
CREATE TABLE AuditLogs (  
  log_id SERIAL PRIMARY KEY,  
  actor_id INT REFERENCES Users(user_id) ON DELETE SET NULL,  
  action VARCHAR(100),  
  entity_type VARCHAR(50),  
  entity_id INT,  
  details JSONB,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

Updated Relationships

- **Users → Students/Companies/Admins** (1–1)
 - **Companies → Jobs** (1–many)
 - **Companies → Rooms** (1–1 on event day)
 - **Companies ↔ Students** via **InterviewRequests** (many–many, students can reject)
 - **InterviewRequests → Interviews** (1–1 after acceptance)
 - **Students → Notifications** (via FCM token)
 - **Companies → Surveys** (post-event feedback)
 - **AuditLogs** track all key actions.
-

Enhancements in this Version

1. **Students can reject interview requests** initiated by companies or other students.
2. **Interview status tracking** for queued, in_progress, shortlisted, hired, rejected.
3. **Queue management** ensures no overlapping interviews; conflict resolution is automatic.
4. **Role-based access:** Students (mobile), Companies (web), Admin (web).
5. **Attendance confirmation** for pre-registered companies via QR + secret key.
6. **Push notifications** via FCM tokens for student updates.



7.

